

Java Backend Architecture

Interview Series

Chapter 8.3

OAuth 2.0 Flows

50+ Interview Q&A

Code Examples

Architecture Diagrams

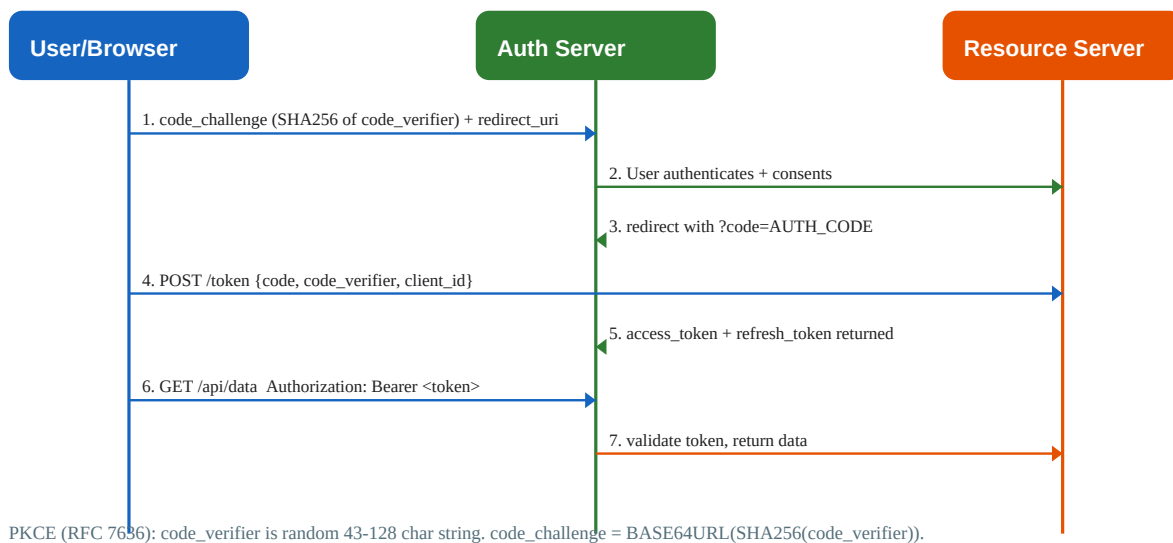
Advanced Topics

Chapter 8.3 — OAuth 2.0 Flows

Introduction

OAuth 2.0 (RFC 6749) is an authorization framework — not authentication. It allows third-party applications to obtain limited access to user resources without exposing credentials. The framework defines multiple grant types for different use cases: web apps, mobile apps, service-to-service, and device flows. PKCE (Proof Key for Code Exchange, RFC 7636) is now mandatory for all public clients.

Authorization Code + PKCE Flow



The Authorization Code flow with PKCE is the recommended flow for web and mobile applications. The client generates a random **code_verifier** and computes **code_challenge = BASE64URL(SHA256(code_verifier))**. The challenge is sent in the authorization request; the verifier is sent when exchanging the code for tokens. This prevents authorization code interception attacks — a stolen code is useless without the verifier.

```
// Spring Security OAuth2 Client configuration @Configuration public class OAuth2Config { @Bean
public SecurityFilterChain filterChain(HttpSecurity http) throws Exception { http
.oauth2Login(oauth2 -> oauth2 .authorizationEndpoint(auth -> auth
.baseUri("/oauth2/authorize")) .redirectionEndpoint(redirect -> redirect
.baseUri("/oauth2/callback/*")) .userInfoEndpoint(userInfo -> userInfo
.userService(customOAuth2UserService)) ); return http.build(); } } # application.yaml - OAuth2
client registration spring: security: oauth2: client: registration: my-auth-server: client-id:
my-client client-secret: ${CLIENT_SECRET} authorization-grant-type: authorization_code
redirect-uri: "{baseUrl}/oauth2/callback/{registrationId}" scope:
openid,profile,email,read:orders client-authentication-method: client_secret_basic provider:
my-auth-server: authorization-uri: https://auth.example.com/oauth/authorize token-uri:
https://auth.example.com/oauth/token user-info-uri: https://auth.example.com/userinfo
jwk-set-uri: https://auth.example.com/.well-known/jwks.json
```

Client Credentials Flow

Used for machine-to-machine (M2M) communication where no user is involved. The client authenticates with its `client_id` and `client_secret` directly to the token endpoint. No user consent or redirect is needed.

Use for microservice-to-microservice calls, batch jobs, background processes.

```
// Spring Boot OAuth2 Resource Server (validates tokens) @Configuration public class
ResourceServerConfig { @Bean public SecurityFilterChain filterChain(HttpSecurity http) throws
Exception { http .authorizeHttpRequests(auth -> auth
.requestMatchers("/api/public").permitAll()
.requestMatchers("/api/orders").hasAuthority("SCOPE_read:orders")
.anyRequest().authenticated()) .oauth2ResourceServer(oauth2 -> oauth2 .jwt(jwt -> jwt
.jwkSetUri("https://auth.example.com/.well-known/jwks.json"))); return http.build(); } } //
Client-side: obtaining a client_credentials token // POST /token Content-Type:
application/x-www-form-urlencoded //
grant_type=client_credentials&client_id=svc-a&client_secret=secret&scope=read:orders //
Response: {"access_token":"eyJ...", "token_type":"Bearer", "expires_in":3600}
```

Token Lifecycle & Management



Access token: expires in 15-60 min. Use refresh token to get new access token (token rotation).

Introspection: POST /introspect {token} -> {active, scope, exp, sub}. Revocation: POST /revoke.

Token rotation: refresh token is single-use. Each use returns a new refresh token, old one invalidated.

Token Type	Lifetime	Format	Storage	Revocable
Access Token	15-60 minutes	JWT or opaque	Memory or HttpOnly cookie	Via introspection or short TTL
Refresh Token	Days to weeks	Opaque string	HttpOnly Secure cookie	Yes — via revocation endpoint
ID Token	Session	JWT (OIDC only)	Memory (client-side)	Via session termination

Grant Types Comparison



Use Auth Code + PKCE for all new applications. Client Credentials for machine-to-machine. Avoid Implicit and ROPC.

Grant Type	Use Case	User Involved?	Redirect?	Status
Authorization Code + PKCE	Web/mobile apps	Yes	Yes	Recommended
Client Credentials	Service-to-service	No	No	Recommended
Device Code	Smart TV/CLI	Yes	No	Recommended
Implicit	Legacy SPA	Yes	Yes	DEPRECATED
ROPC	Legacy trusted app	Yes	No	DEPRECATED
Refresh Token	Token renewal	No	No	Extension grant

Scopes

Scopes define the granular permissions a client is requesting. The resource owner (user) approves scopes at the consent screen. Common OIDC scopes: **openid** (required for OIDC), **profile**, **email**. Custom scopes: **read:orders**, **write:orders**, **admin**. In Spring Security, scopes become authorities prefixed with `SCOPE_`.

Scope	Purpose	Grants Access To
openid	OIDC identifier	ID token, sub claim
profile	Basic profile info	name, picture, locale
email	Email address	email, email_verified
read:orders	Custom — read orders	/api/orders GET
write:orders	Custom — write orders	/api/orders POST/PUT
offline_access	Refresh token	Long-lived refresh token

Token Introspection & Revocation

```
// Token introspection (RFC 7662) – resource server validates opaque token // POST /introspect
// Authorization: Basic <client_credentials> // token=ACCESS_TOKEN_TO_CHECK // // Response
(active): //
{"active":true,"sub":"user123","scope":"read:orders","exp":1700000000,"client_id":"app1"} //
// Response (invalid): // {"active":false} // Spring: configure opaque token validation
http.oauth2ResourceServer(oauth2 -> oauth2 .opaqueToken(opaque -> opaque
.introspectionUri("https://auth.example.com/introspect")
.introspectionClientCredentials("resource-server", "secret"))); // Token revocation (RFC 7009)
// POST /revoke // token=REFRESH_TOKEN_TO_REVOKE&token_type_hint=refresh_token
```

50+ Interview Questions & Answers

Q1. What is OAuth 2.0?

An authorization framework (RFC 6749) that allows third-party applications to obtain limited access to user resources via delegated authorization. It is NOT an authentication protocol — it issues access tokens, not identity assertions.

Q2. What are the OAuth 2.0 roles?

Resource Owner (user), Client (application requesting access), Authorization Server (issues tokens), Resource Server (hosts protected resources). In many setups, Auth Server and Resource Server are different services.

Q3. Why is PKCE required for public clients?

Public clients (SPAs, mobile apps) cannot securely store a `client_secret`. PKCE replaces the secret with a per-request `code_verifier/code_challenge` pair, preventing authorization code interception attacks.

Q4. What is a code_verifier and code_challenge?

`code_verifier`: a cryptographically random string (43-128 chars). `code_challenge` = `BASE64URL(SHA256(code_verifier))`. Challenge is sent with auth request; verifier is sent when exchanging code for tokens. Server recomputes and verifies.

Q5. What is the difference between OAuth 2.0 and OAuth 1.0?

OAuth 1.0 required complex request signing (HMAC-SHA1). OAuth 2.0 relies on TLS for security and uses bearer tokens. OAuth 2.0 is simpler and more flexible with multiple grant types.

Q6. What is the Authorization Code grant flow?

1) Client redirects user to Auth Server with `client_id`, `redirect_uri`, `scope`, `state`, `code_challenge`. 2) User authenticates + consents. 3) Auth Server redirects back with authorization code. 4) Client exchanges code for tokens via back-channel POST.

Q7. What is the Client Credentials grant?

Client authenticates with `client_id` + `client_secret` (or client assertion) directly to `/token` endpoint. No user involvement. Returns access token only (no refresh token by default). Used for M2M communication.

Q8. Why is the Implicit grant deprecated?

Tokens were returned in the URL fragment (visible in browser history, referrer headers, logs). No back-channel exchange — no way to keep tokens secret. Replaced by Auth Code + PKCE which keeps tokens in back-channel.

Q9. Why is ROPC (Resource Owner Password Credentials) deprecated?

Client collects username/password directly — defeats OAuth's purpose of not sharing credentials with clients. Cannot support MFA or SSO. Only acceptable for trusted first-party apps, but even then use device flow or auth code.

Q10. What is the state parameter in OAuth?

A random value (CSRF token) the client generates and includes in the authorization request. Auth server returns it unchanged. Client validates state matches — prevents CSRF attacks on the callback endpoint.

Q11. What is the difference between access token and refresh token?

Access token: short-lived (15-60 min), used to call APIs. Refresh token: long-lived (days/weeks), used to get new access tokens without re-authentication. Refresh tokens must be stored securely (HttpOnly cookies).

Q12. What is token rotation?

When a refresh token is used, the auth server issues a new refresh token and invalidates the old one. If the old token is used again (replay attack), the auth server can detect and revoke all tokens in the family.

Q13. What is token introspection?

RFC 7662 — resource server calls the auth server's `/introspect` endpoint to validate an opaque token. Response includes `active`, `sub`, `scope`, `exp`, `client_id`. Used when tokens are opaque (not JWT).

Q14. What is token revocation?

RFC 7009 — client calls `/revoke` endpoint with a token to invalidate it. Used on logout, password change, security breach. Works for both access and refresh tokens (hint with `token_type_hint`).

Q15. What is a JWT access token?

A self-contained token the resource server can validate locally without calling the auth server. Contains `header.payload.signature`. Resource server verifies signature against auth server's public key (JWKS endpoint).

Q16. What is the JWKS endpoint?

`/.well-known/jwks.json` — the auth server's public key set in JSON Web Key Set format. Resource servers fetch this to validate JWT signatures. Keys have a `kid` (key ID) that matches the JWT header's `kid`.

Q17. What is token binding?

Binding a token to a specific TLS connection or device, preventing token theft. Draft RFC not widely adopted. Practical alternative: short token lifetime + refresh token rotation + detecting concurrent refresh.

Q18. What are scopes in OAuth 2.0?

Strings that define the permissions a client is requesting. The user approves scopes at the consent screen. Resource server enforces scope-based access. Examples: read:orders, write:users, openid, profile.

Q19. How does Spring Security handle OAuth2 resource server?

Add `spring-security-oauth2-resource-server` dependency. Configure `.oauth2ResourceServer().jwt()` with `jwt-set-uri`. Spring auto-validates JWT signature, expiry, and issuer. Scopes map to authorities with `SCOPE_` prefix.

Q20. What is the difference between OAuth 2.0 and OIDC?

OAuth 2.0: authorization only — grants access to resources. OIDC: adds identity layer on top of OAuth 2.0 — provides ID token (JWT with user identity), `UserInfo` endpoint, and standard claims (sub, email, name).

Q21. What is the device authorization grant?

RFC 8628 — for devices without browsers (Smart TV, CLI). Device displays user code + verification URL. User visits URL on phone/computer, enters code, authenticates. Device polls `/token` until approved.

Q22. What is pushed authorization request (PAR)?

RFC 9126 — client POSTs authorization parameters directly to auth server (instead of URL), gets `request_uri`. Prevents parameter tampering and keeps parameters server-side. Useful for complex/sensitive authorization requests.

Q23. What is Rich Authorization Requests (RAR)?

RFC 9396 — allows clients to specify fine-grained `authorization_details` in the authorization request (e.g., specific account numbers, transaction amounts). More expressive than simple scopes.

Q24. What is the difference between access token as JWT vs opaque?

JWT: self-contained, resource server validates locally (fast, no network call). Opaque: random string, resource server must call introspection endpoint (adds latency but enables real-time revocation). Many systems use JWT with short expiry + introspection for revocation.

Q25. How do you handle OAuth2 security in microservices?

API gateway validates tokens before routing. Downstream services also validate tokens (defense-in-depth). Use mTLS between services. Pass user context via JWT claims — avoid re-authentication at each service. Use token relay pattern.

Q26. What is the token relay pattern?

In Spring, the `OAuth2TokenRelayFilter` or `WebClient` with `ServerOAuth2AuthorizedClientExchangeFilterFunction` propagates the incoming access token to downstream service calls — same token is used across the service chain.

Q27. What is a confidential vs public client?

Confidential client: can securely store a `client_secret` (server-side web apps, backend services). Public client: cannot securely store secrets (SPAs, mobile apps, desktop apps). Public clients must use PKCE.

Q28. What is front-channel vs back-channel in OAuth?

Front-channel: communication via browser redirects (URL parameters) — visible, not fully trusted. Back-channel: direct server-to-server HTTP calls — private, trusted. Code exchange happens back-channel for security.

Q29. How do you prevent CSRF in OAuth callbacks?

Use the state parameter: generate a random state before redirecting, store in session, verify it matches when callback arrives. Spring Security OAuth2 does this automatically.

Q30. What is the authorization server metadata endpoint?

RFC 8414 — /.well-known/oauth-authorization-server. Returns JSON with all endpoints: authorization_endpoint, token_endpoint, introspection_endpoint, revocation_endpoint, jwks_uri, grant_types_supported.

Q31. What is dynamic client registration?

RFC 7591 — clients can register dynamically with the auth server via a registration endpoint. Auth server issues client_id and optionally client_secret. Used in federation and open ecosystems.

Q32. How do you implement rate limiting on OAuth token endpoints?

Apply rate limiting at the auth server: per client_id for /token, per IP for /authorize. Token endpoint brute force protection: lock client after N failed attempts. Use Redis-backed rate limiting.

Q33. What is the difference between authorization server and identity provider?

Authorization server manages OAuth 2.0 token issuance. Identity Provider (IdP) manages user identities and authentication. Often combined (Keycloak, Auth0 act as both). OIDC bridges them — auth server issues ID tokens with identity claims.

Q34. What is Keycloak?

Open-source IAM (Identity and Access Management) server. Supports OAuth 2.0, OIDC, SAML 2.0. Features: user federation, social login, admin console, multi-realm, client scopes, composite roles, client policies.

Q35. What is Auth0?

Cloud-based IAM platform. Supports OAuth 2.0, OIDC, SAML. Features: universal login, custom rules/actions, machine-to-machine, organizations, anomaly detection. Uses client_id/client_secret for confidential clients.

Q36. What is the nonce parameter in OAuth/OIDC?

A random value included in the authorization request and echoed in the ID token. Client validates nonce in ID token matches what was sent — prevents replay attacks on ID tokens. Required for implicit flow (mitigated risk).

Q37. What is offline_access scope?

Requests a refresh token. Some auth servers only issue refresh tokens when offline_access scope is explicitly requested. Indicates the client needs to access resources when the user is not present.

Q38. What is a JWT bearer grant?

RFC 7523 — client uses a signed JWT as client authentication or as an authorization grant (replacing client_secret). The client signs a JWT with its private key; auth server verifies with client's public key. Used in high-security M2M.

Q39. What is the difference between JWT and JWS, JWE, JWK?

JWT: the overall format. JWS (JSON Web Signature): signed JWT (most common). JWE (JSON Web Encryption): encrypted JWT. JWK (JSON Web Key): key representation format. JOSE is the umbrella standard covering all.

Q40. What are JWT validation steps?

1) Parse header, verify alg is expected (reject 'none'). 2) Verify signature against issuer's public key (JWKS). 3) Verify exp (not expired). 4) Verify iss (issuer matches expected). 5) Verify aud (audience includes this service). 6) Verify nbf if present.

Q41. What is the alg:none attack on JWT?

Attacker modifies JWT header to alg:'none' and removes signature. Vulnerable libraries accept the token. Prevention: whitelist expected algorithms (RS256, ES256) — never accept 'none'. Spring Security rejects 'none' by default.

Q42. How do you implement OAuth2 in Spring Boot Authorization Server?

Use spring-authorization-server library. Configure RegisteredClientRepository (in-memory or JDBC), UserDetailsService, JWKSsource (RSA key pair for signing), AuthorizationServerSettings. Exposes /oauth2/authorize, /oauth2/token, /oauth2/jwks endpoints.

Q43. What is consent in OAuth?

The screen shown to users to approve the requested scopes. Allows users to see what permissions the app is requesting. Auth server stores consent to avoid re-prompting. Spring Authorization Server shows consent screen by default.

Q44. What is the difference between grant_type=password and grant_type=client_credentials?

Password grant: client collects user credentials and exchanges for tokens (user context, deprecated). Client credentials: client authenticates itself (no user context, service-to-service, still valid).

Q45. What is a software statement in OAuth?

A signed JWT issued by a trust authority that asserts metadata about an OAuth client. Used in open banking (FAPI) and dynamic client registration to prove client identity and metadata integrity.

Q46. What is FAPI (Financial-grade API)?

An OAuth 2.0 security profile for high-value APIs. FAPI 1.0 adds PKCE, state, nonce requirements. FAPI 2.0 requires PAR, DPoP (Demonstrating Proof of Possession). Used in open banking implementations.

Q47. What is DPoP (Demonstrating Proof of Possession)?

RFC 9449 — access tokens are bound to the client's public key. Client proves key possession by signing each request with the matching private key. Prevents token theft — stolen token is useless without the private key.

Q48. What is mTLS client authentication in OAuth?

RFC 8705 — client authenticates to /token endpoint using its TLS client certificate instead of client_secret. Token can be bound to client certificate fingerprint. Used in banking and high-security M2M.

Q49. What is the difference between resource server and client in OAuth?

Client: application that wants to access resources (obtains tokens). Resource server: API server that holds protected resources (validates tokens). Same application can act as both (e.g., a BFF that fetches its own resources on behalf of users).

Q50. What is Back-end For Frontend (BFF) pattern in OAuth?

Server-side component handles OAuth flow for SPAs. BFF holds tokens server-side, SPA gets a session cookie. Prevents token exposure in browser. BFF proxies API calls with token. Recommended by IETF OAuth for Browser-Based Apps draft.

Q51. How do you test OAuth2 flows?

Use Postman with OAuth 2.0 authorization type. Test access token endpoint with curl. For resource server: WireMock to mock JWKS endpoint, TestRestTemplate with real tokens. Spring Security test: @WithMockUser or MockMvcRequestPostProcessors.jwt().

Q52. What is the exp claim in JWT?

Expiration Time — Unix timestamp after which the token must not be accepted. Resource server MUST reject expired tokens. Typical: iat + 3600 (1 hour). Clock skew tolerance: usually 30-60 seconds allowed (leeway parameter in Spring).